VIT-AP UNIVERSITY

Computer Networks | Project Report

RUFT

Reliable UDP File Transfer Protocol

A custom network protocol with a Go-Back-N sliding window

Submitted by	**Priya Ranjan Pradhan**
Registration No.	**24BCE7879**
Course	**Computer Networks (CSE3003)**
Faculty	**D. Santhadevi**
Class slot	**E1 + TE1**
Semester	**Fall 2026-27**
Submission date	**31 October 2026**

GitHub: github.com/PriyaRanjan018/ruft-protocol

Abstract

UDP is fast and simple, but it gives an application no guarantee that its data arrives. Packets can be lost, damaged, duplicated or reordered, and neither end is told. This project designs and implements **RUFT (Reliable UDP File Transfer)**, an application-layer protocol that delivers a complete, verified file across such a network.

RUFT defines its own 11-byte packet header, a three-phase session of handshake, data transfer and finish, and a Go-Back-N sliding window with cumulative acknowledgements and timeout-based retransmission. Damaged packets are detected with CRC-32, and the finished file is verified end to end with SHA-256. Setting the window size to 1 turns the same code into stop-and-wait, which allows the two methods to be compared directly.

The protocol was implemented in about 360 lines of Python using only the standard library, with a built-in simulator for packet loss and corruption. All 22 automated tests pass. A 2 MB file was delivered byte-identical at 15% loss and 5% corruption in both directions. In benchmarks, a window of 4 was about twice as fast as stop-and-wait under loss, while larger windows only added retransmissions. Stop-and-wait retransmission counts matched a probability model to within 7%.

Keywords: network protocol, UDP, reliable data transfer, Go-Back-N, sliding window, automatic repeat request, CRC-32, SHA-256, socket programming

22 / 22	11 bytes	2.0x	2 MB
automated tests passing	custom packet header	faster than stop-and-wait at 5% loss, using window 4	delivered intact at 15% loss and 5% corruption

> **Source code**
> The complete code, tests and benchmark data are public at https://github.com/PriyaRanjan018/ruft-protocol

Contents

Abstract **2**

1 Introduction **4**

1.1 Background 4
1.2 Problem statement 4
1.3 Objectives 4
1.4 Scope 4

2 Background Study **5**

2.1 Why UDP needs help 5
2.2 Automatic repeat request 5
2.3 Error detection 5
2.4 Related protocols 5

3 Protocol Design **6**

3.1 Position in the network stack 6
3.2 Packet format 6
3.3 Session phases 7
3.4 Go-Back-N sliding window 8
3.5 Failure handling 9
3.6 Key design decisions 9

4 Implementation **10**

4.1 Tools and code structure 10
4.2 Building and checking packets 10
4.3 The sender 11
4.4 The receiver 12
4.5 Simulating an unreliable network 12
4.6 Input validation 13

5 Testing **14**

5.1 Automated tests 14
5.2 Checking the tests themselves 14
5.3 End-to-end test 14
5.4 A real recovery, packet by packet 15

6 Results and Analysis **16**

6.1 Method 16
6.2 Throughput 16
6.3 The cost of a large window 17
6.4 Comparison with theory 17
6.5 Limits of these results 18

7 Running the Project **19**

7.1 Basic transfer 19
7.2 Demonstrating recovery 19
7.3 Other commands 19

8 Limitations and Future Work **20**

9 Conclusion **20**

References **21**

1 Introduction

1.1 Background

A network protocol is a set of rules that two computers agree on so they can communicate. Every protocol defines three things: **syntax**, the format of each message; **semantics**, what each message means; and **timing**, the order of messages and what to do when something goes wrong.

The internet's transport layer offers two main services. TCP provides reliable, ordered delivery but handles all of it inside the operating system. UDP simply sends individual packets with no guarantees. Applications that want different trade-offs can build their own reliability rules on top of UDP, which is how protocols such as TFTP and QUIC work.

1.2 Problem statement

A file must arrive complete and unchanged, or the sender must be told that it failed. UDP alone cannot provide this. If one packet out of thousands disappears, the receiver ends up with a silently corrupted file. The problem is to design rules that turn this unreliable service into a reliable one, and to show by experiment that they work.

1.3 Objectives

- Define a complete protocol: packet format, session phases and failure handling.
- Implement it from scratch, without using TCP for reliability or any external library.
- Recover automatically from lost, damaged, duplicated and out-of-order packets.
- Verify the whole file end to end, and never keep a file that fails verification.
- Compare stop-and-wait with Go-Back-N under controlled packet loss.

1.4 Scope

RUFT transfers one file at a time from a client to a server over IPv4. It is a teaching and demonstration protocol, so encryption, congestion control and parallel transfers are deliberately left out. Section 8 lists these limits.

2 Background Study

2.1 Why UDP needs help

UDP adds only port numbers and an optional checksum to IP. Without acknowledgements, the sender never learns whether a packet arrived. Without sequence numbers, the receiver cannot detect gaps, repeats or reordering.

Table 1 Transport services compared with RUFT

Feature	UDP	TCP	RUFT
Delivery guarantee	None	Yes	Yes, retransmission
Ordering	None	Yes	Yes, sequence numbers
Error detection	Optional checksum on IPv4	16-bit checksum	CRC-32 on each packet, SHA-256 on the file
Congestion control	None	Yes	None

2.2 Automatic repeat request

Reliable delivery normally uses automatic repeat request (ARQ): the receiver acknowledges what it gets, and the sender resends anything not acknowledged in time. There are three classic forms:

- **Stop-and-wait:** send one packet, then wait for its ACK. Simple, but the link sits idle during every wait.
- **Go-Back-N:** keep up to N packets unacknowledged, and on a timeout, resend from the oldest one. The receiver only accepts packets in order.
- **Selective repeat:** resend only the missing packets. This is the most efficient of the three, but the receiver must buffer out-of-order packets.

RUFT uses Go-Back-N because the receiver stays simple, and a window of 1 reduces it to stop-and-wait, so both can be tested with the same code.

2.3 Error detection

CRC-32 is a fast 32-bit check that is very effective at detecting accidental damage, so RUFT applies it to every packet. **SHA-256** is a cryptographic hash, used here on the whole file. Passing CRCs do not prove the file was reassembled correctly, but matching hashes do.

> **Important limit**
> Neither check provides security here. There is no secret key, so an attacker who can alter packets could also recompute the checksum and the hash. They protect against accidents, not attackers.

2.4 Related protocols

TFTP (RFC 1350) transfers files over UDP using stop-and-wait with 512-byte blocks. **QUIC** (RFC 9000) builds a full encrypted transport on UDP for HTTP/3. RUFT sits between them, adding a sliding window and a whole-file check that TFTP lacks.

3 Protocol Design

3.1 Position in the network stack

RUFT is an application-layer protocol. As Figure 1 shows, it uses UDP as a basic pipe and adds all of the reliability itself.

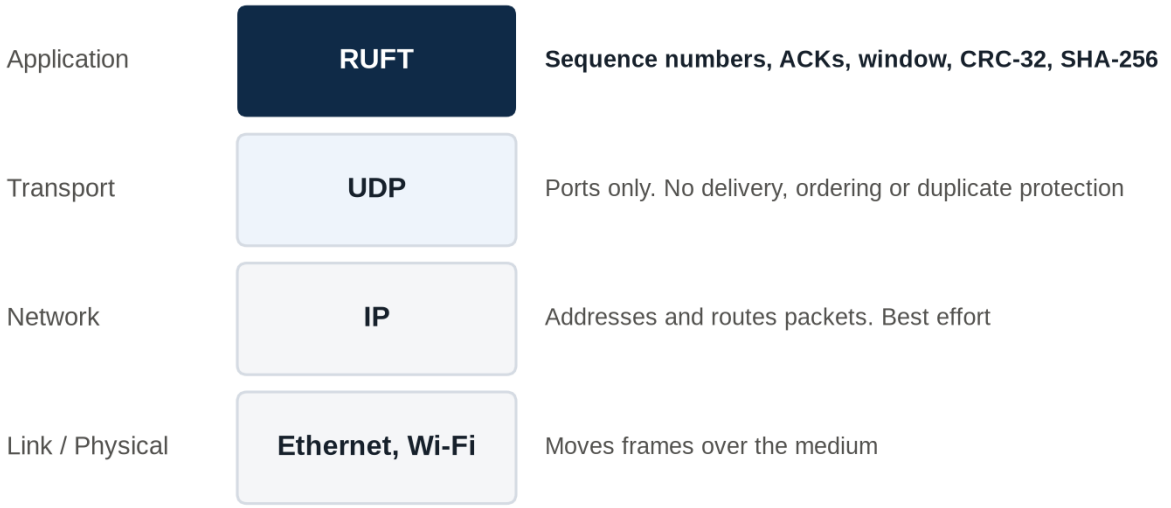


Figure 1 Where RUFT sits in the network stack

3.2 Packet format

Every packet starts with the same 11-byte header in network byte order, followed by up to 1024 bytes of data. Figure 2 shows the layout and the actual bytes of one packet produced by the code.

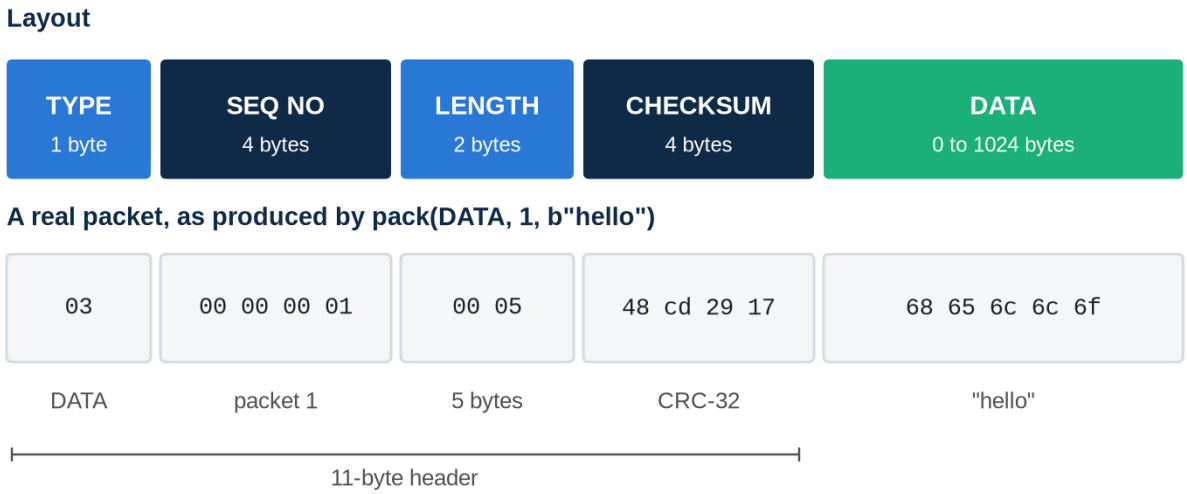


Figure 2 Packet layout, with a real 16-byte packet carrying the text "hello"

Table 2 Header fields

Field	Size	Purpose
TYPE	1 byte	`SYN` = 1, `ACK` = 2, `DATA` = 3, `FIN` = 4
SEQ NO	4 bytes	Packet number. The SYN and its ACK use the reserved value 0xFFFFFFFF
LENGTH	2 bytes	Number of data bytes that follow
CHECKSUM	4 bytes	CRC-32 over the header (with this field set to zero) and the data

3.3 Session phases

A transfer has three phases, shown in Figure 3. Each one retries on its own, so a lost control packet cannot stall the session forever.

- **Handshake.** The client sends `SYN` with the file's name, size and SHA-256 hash as JSON. The server prepares a temporary `.part` file and replies with `ACK`.
- **Data transfer.** The file is split into 1024-byte `DATA` packets numbered from 0, and sent using the sliding window.
- **Finish.** The client sends `FIN`. The server compares the SHA-256 hash, then replies with an `ACK` containing `OK` or `HASH_MISMATCH`.

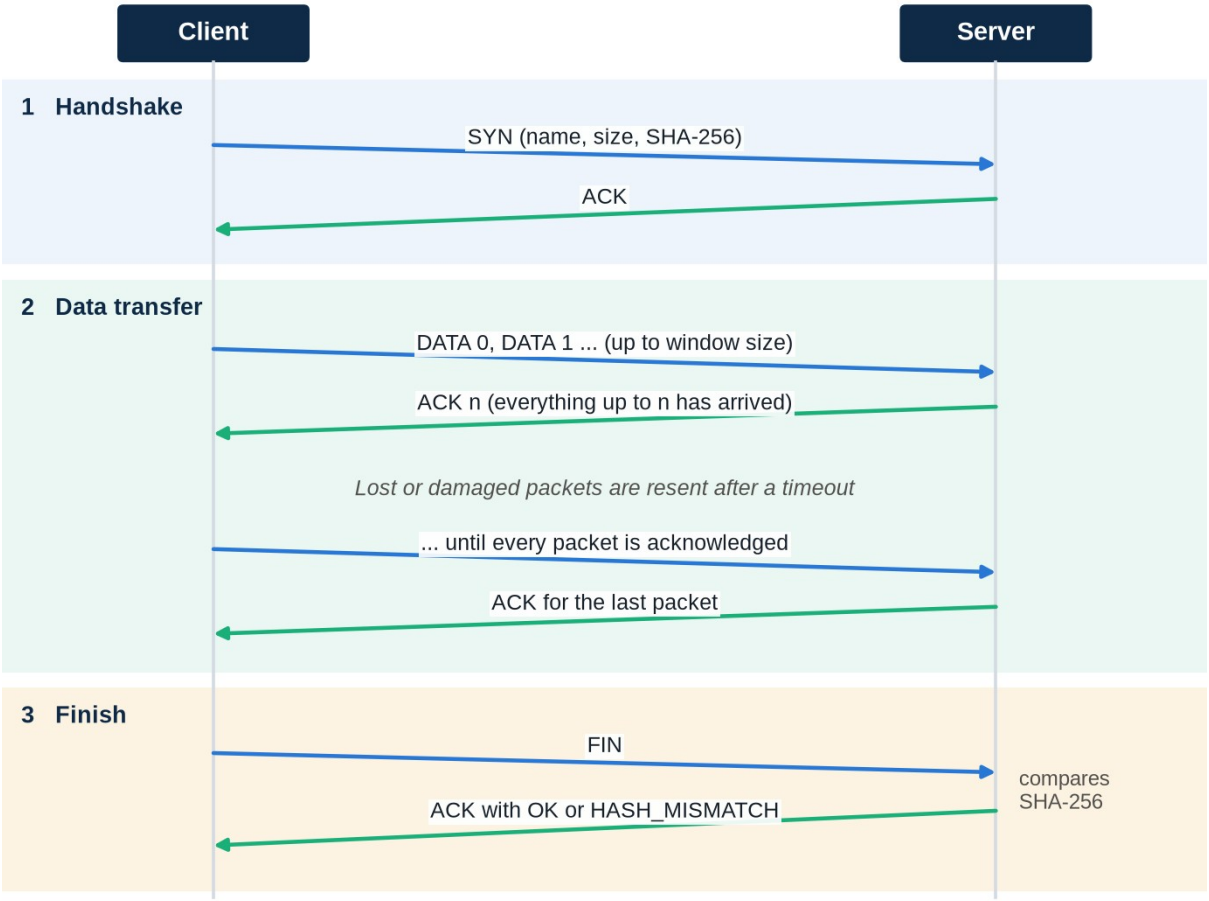


Figure 3 The three phases of a RUFT session

3.4 Go-Back-N sliding window

The sender keeps two positions, shown in Figure 4: `base`, the oldest unacknowledged packet, and `next_seq`, the next packet to send. It may send until `next_seq` is a full window ahead of `base`.

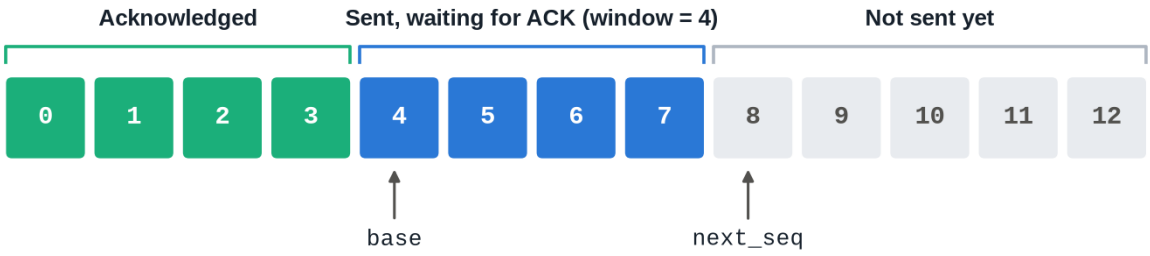


Figure 4 Sliding window state with a window size of 4

ACKs are **cumulative**: `ACK n` means every packet up to and including `n` has arrived, so `base` moves to `n + 1`. A lost ACK therefore does no harm if a later one arrives.

Figure 5 shows a loss. The receiver only accepts the packet it expects, so packets 2 and 3 are discarded even though they arrived safely. When the timer runs out, the sender goes back and resends all three. That is the cost of keeping the receiver simple.

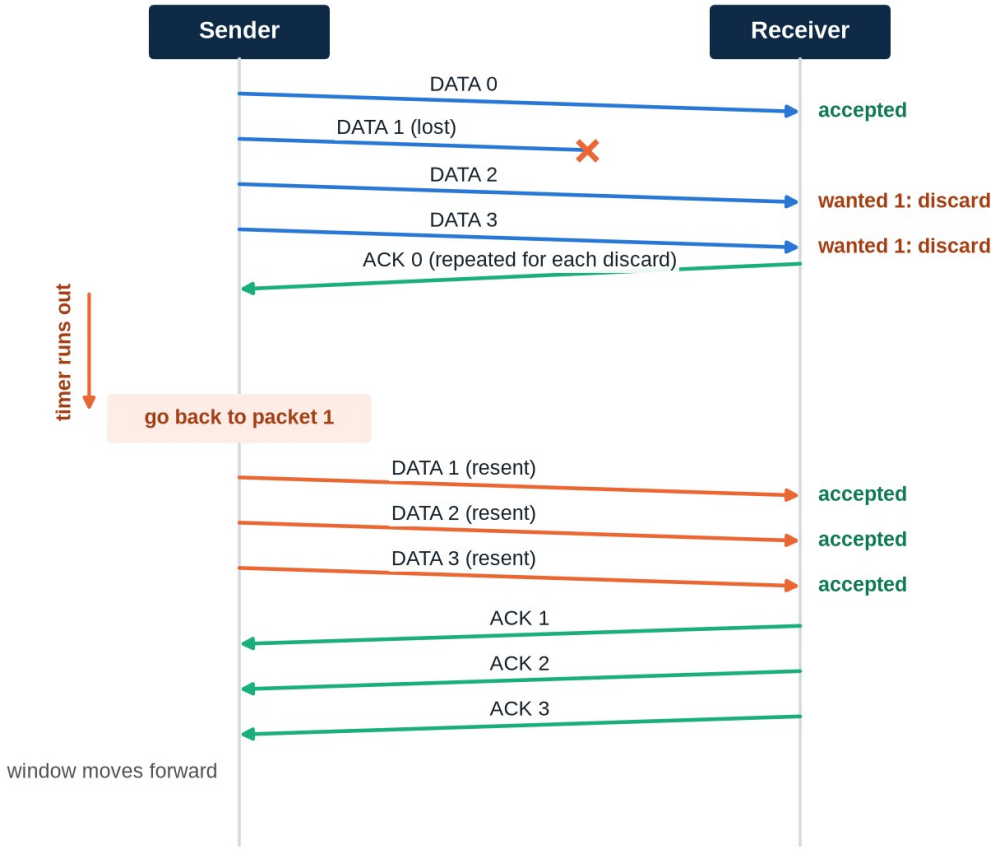


Figure 5 Recovering from one lost packet with a window of 4

3.5 Failure handling

Table 3 How RUFT responds to each failure

Failure	Detected by	Response
Packet lost	No ACK before timeout	Resend from the oldest unacknowledged packet
Packet damaged	CRC-32 mismatch	Discard, so it is handled as a lost packet
ACK lost	Later ACK or timeout	A later cumulative ACK covers it, or the duplicate data is discarded
Duplicate packet	Sequence number	Discard and repeat the last good ACK
Out-of-order packet	Sequence number	Discard and repeat the last good ACK
Wrong final file	SHA-256 mismatch	Delete the file and report `HASH_MISMATCH`
Final ACK lost	Client timeout	Client resends `FIN`, and the server repeats the stored result
Server unreachable	20 failed tries	Client stops and reports failure
Client disappears	10 seconds idle	A new client's `SYN` replaces the session, and the partial file is deleted

3.6 Key design decisions

- **Reserved sequence number for the handshake.** If the SYN's ACK used sequence 0, a delayed copy could be mistaken for `ACK 0` and the sender would skip a packet. Using 0xFFFFFFFF makes that impossible.
- **Write to a temporary file.** Data goes to a `.part` file, which gets its real name only after the hash matches, so a failed transfer never leaves a bad file behind.
- **Store the final result.** The server remembers `OK` or `HASH_MISMATCH`, so a repeated `FIN` gets the same answer.
- **Timer resets only on progress.** A stream of repeated ACKs cannot keep extending the timeout.

4 Implementation

4.1 Tools and code structure

RUFT is written in Python 3 and uses only the standard library: `socket` for UDP, `struct` for the binary header, `zlib` for CRC-32 and `hashlib` for SHA-256. There is nothing to install. Figure 6 shows how the files depend on each other.

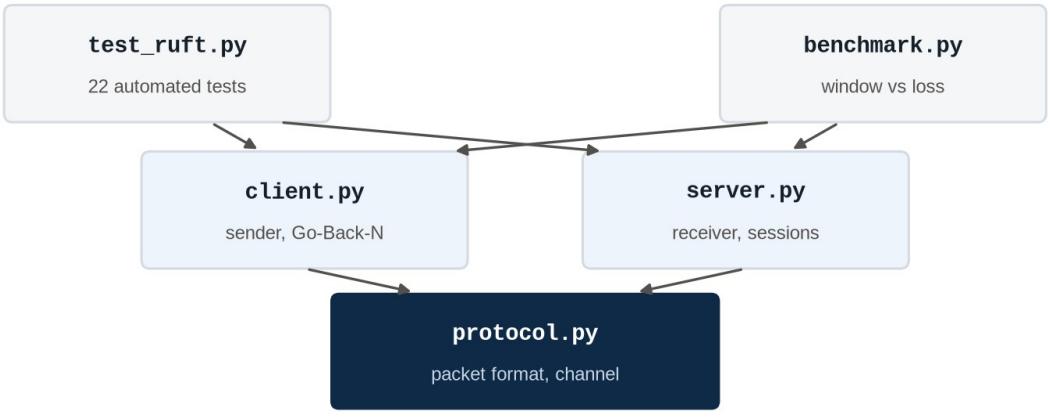


Figure 6 Code structure. Arrows point from a file to the files it imports

Table 4 Source files

File	Lines	Purpose
`protocol.py`	80	Packet format, checksum, failure-simulating channel
`client.py`	134	Sender: handshake, sliding window, finish, statistics
`server.py`	144	Receiver: sessions, ordering, verification, input checks
`test_ruft.py`	176	22 automated tests
`benchmark.py`	39	Window size against loss rate measurements

4.2 Building and checking packets

`pack` calculates the CRC-32 with the checksum field set to zero, then places the result in the header. `unpack` repeats the calculation and returns `None` for anything damaged, so the rest of the code never sees a bad packet.

Listing 1 Packet construction and validation (protocol.py)

```
def pack(ptype, seq, data=b""):
    if len(data) > MAX_DATA:
        raise ValueError(f"payload of {len(data)} bytes exceeds {MAX_DATA}")
    crc = zlib.crc32(HEADER.pack(ptype, seq, len(data), 0) + data)
    return HEADER.pack(ptype, seq, len(data), crc) + data


def unpack(raw):
    """Return (type, seq, data), or None if the packet is damaged."""
    if len(raw) < HEADER.size:
        return None
    ptype, seq, length, crc = HEADER.unpack_from(raw)
    data = raw[HEADER.size:]
    if ptype not in NAMES or length != len(data):
        return None
    if zlib.crc32(HEADER.pack(ptype, seq, length, 0) + data) != crc:
        return None
    return ptype, seq, data
```

4.3 The sender

This function is the core of the protocol. Each pass fills the window, then waits for one ACK or for the timer. A valid ACK slides the window forward. When the timer expires, setting `next_seq = base` makes the next pass resend everything still unacknowledged.

Listing 2 Go-Back-N sender (client.py)

```
def send_window(chan, addr, f, total, window, timeout):
    """Go-Back-N: keep up to `window` packets in flight, resend all on timeout.

    Returns (retransmits, timeouts). window=1 is stop-and-wait.
    """
    base = next_seq = highest = 0  # oldest unACKed, next to send, next never-sent
    retransmits = timeouts = tries = 0
    deadline = time.monotonic() + timeout
    while base < total:
        while next_seq < min(base + window, total):
            f.seek(next_seq * MAX_DATA)
            resend = next_seq < highest
            retransmits += resend
            chan.send(DATA, next_seq, f.read(MAX_DATA), addr, "RETRANSMIT" if resend else "SEND")
            next_seq += 1
            highest = max(highest, next_seq)

        ack = recv_ack(chan, addr, max(deadline - time.monotonic(), 0.0001))
        if ack and base <= ack[0] < next_seq:  # cumulative: everything up to ack[0] arrived
            base = ack[0] + 1
            tries = 0
            deadline = time.monotonic() + timeout
        elif time.monotonic() >= deadline:
            tries += 1
            timeouts += 1
            if tries >= MAX_RETRIES:
                raise TransferError(f"connection lost at packet {base}")
            chan.log.debug("TIMEOUT: going back to seq=%d", base)
            next_seq = base
            deadline = time.monotonic() + timeout
    return retransmits, timeouts
```

4.4 The receiver

The receiver follows the logic in Figure 7. It stores only the next expected packet, which is why it needs no buffer for out-of-order data.

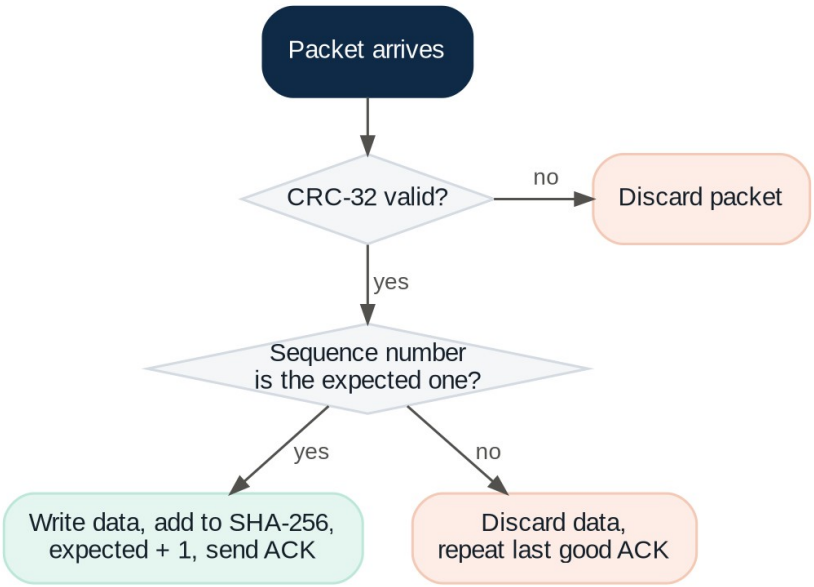


Figure 7 How the receiver handles each DATA packet

Listing 3 Receiver logic (server.py)

```
def handle_transfer(chan, s, ptype, seq, data):
    if ptype == DATA:
        if seq == s.expected and seq < s.total:
            s.write(data)
            chan.send(ACK, seq, b"", s.addr)
        elif s.expected > 0:  # duplicate or out of order: repeat the last good ACK
            chan.send(ACK, s.expected - 1, b"", s.addr, "RE-ACK")
    elif ptype == FIN and seq == s.total and s.expected == s.total:
        if s.result is None:
            s.finish()
            chan.log.info("%s from %s:%d: %s", s.path.name, *s.addr, s.result.decode())
        chan.send(ACK, seq, s.result, s.addr)
```

4.5 Simulating an unreliable network

A local machine rarely loses packets, so the `Channel` class creates failures on purpose. The `--loss` and `--corrupt` options set the chance of dropping or damaging each outgoing packet, and `--seed` makes a run repeatable. Damage is a real flipped byte, so the CRC-32 check does the actual detection.

Listing 4 Failure simulation (protocol.py)

```
    def send(self, ptype, seq, data, addr, label="SEND"):
        raw = bytearray(pack(ptype, seq, data))
        self.sent += 1
        if self.rng.random() < self.loss:
            self.dropped += 1
            self.log.debug("%s %s seq=%d -> LOST (simulated)", label, NAMES[ptype], seq)
            return
        if self.rng.random() < self.corrupt:
            self.corrupted += 1
            raw[self.rng.randrange(len(raw))] ^= 0xFF
            self.log.debug("%s %s seq=%d -> CORRUPTED (simulated)", label, NAMES[ptype], seq)
        else:
            self.log.debug("%s %s seq=%d", label, NAMES[ptype], seq)
        self.sock.sendto(raw, addr)
```

4.6 Input validation

The server treats everything from the network as untrusted. The file name in a `SYN` is reduced to its last component, so a name such as `../../evil.txt` cannot write outside the output folder. Invalid JSON, wrong field types, negative sizes and unsafe names are ignored without crashing the server. The server also listens only on localhost unless told otherwise.

5 Testing

5.1 Automated tests

The suite uses Python's built-in `unittest` and runs real client and server code over real UDP sockets on the loopback interface. The tests were written before the implementation. Running `python3 -m unittest` gives **22 tests, all passing, in about 1.3 seconds**.

Table 5 Test coverage

Area	Tests	What is checked
Packet format	5	Header size, round trip, every flipped byte rejected, truncated and oversized packets
File transfers	8	Clean network, stop-and-wait, empty file, exact packet-size boundary, loss and corruption both ways, lost ACKs, spurious retransmits
Failure	1	Client gives up when no server answers
Protocol rules	8	Path traversal, invalid and negative SYNs, wrong hash, repeated FIN, out-of-order data, busy server, abandoned session

5.2 Checking the tests themselves

Passing tests only matter if they can fail. So four bugs were inserted on purpose, one at a time, and the suite was run against each. Every bug was caught.

Table 6 Deliberate bugs and the result

Bug inserted	Part affected	Tests failed
Skip the CRC-32 check	Error detection	2
Accept out-of-order data	Receiver ordering	3
No go-back on timeout	Retransmission	3
Trust the client's file name	Security check	1

5.3 End-to-end test

A 2,000,000-byte random file was sent between separate client and server processes, with both sides set to 15% loss and 5% corruption, a window of 8 and a 20 ms timeout. The Linux `cmp` tool confirmed that the received file was identical.

Table 7 End-to-end run, client-side counts

Measure	Value
Data packets in the file	1,954
Total packets sent	5,918
Packets retransmitted	3,959
Timeouts	495
Packets dropped by the simulator	916
Packets damaged by the simulator	263
Time	10.26 s
Result	**Identical, verified**

The suite was also run 15 times in a row to check for timing problems, with no failures. Failure paths were checked by hand as well: with no server running, or with a missing file, the client prints a clear message and exits with an error code.

5.4 A real recovery, packet by packet

Listing 5 is the unedited client log of an 8-packet transfer with a window of 4. Packets 4 and 7 were lost. The server's log shows that `ACK 2` was lost as well, but this caused no resend because the cumulative `ACK 3` covered it. The sender then timed out, went back to packet 4 and finished. The received file matched.

Listing 5 Client log of a real lossy transfer

```
[client] SEND SYN seq=4294967295
[client] RECV ACK seq=4294967295
[client] SEND DATA seq=0
[client] SEND DATA seq=1
[client] SEND DATA seq=2
[client] SEND DATA seq=3
[client] RECV ACK seq=0
[client] SEND DATA seq=4 -> LOST (simulated)
[client] RECV ACK seq=1
[client] SEND DATA seq=5
[client] RECV ACK seq=3
[client] SEND DATA seq=6
[client] SEND DATA seq=7 -> LOST (simulated)
[client] RECV ACK seq=3
[client] TIMEOUT: going back to seq=4
[client] RETRANSMIT DATA seq=4
[client] RETRANSMIT DATA seq=5
[client] RETRANSMIT DATA seq=6
[client] RETRANSMIT DATA seq=7
[client] RECV ACK seq=4
[client] RECV ACK seq=5
[client] RECV ACK seq=6
[client] RECV ACK seq=7
[client] SEND FIN seq=8
[client] RECV ACK seq=8
```

6 Results and Analysis

6.1 Method

`benchmark.py` sent a 500 KB file (489 data packets) with four window sizes and four loss rates. Each setting ran 3 times with different random seeds, and the table shows the median. Loss applied in both directions, the timeout was 20 ms, and everything ran over the loopback interface on one machine using Python 3.12 on Linux.

Table 8 Benchmark results (median of 3 runs)

Window	Loss	Time (s)	Throughput (KB/s)	Retransmitted
1 (stop-and-wait)	0%	0.013	38,708	0
1 (stop-and-wait)	5%	1.075	465	51
1 (stop-and-wait)	10%	2.428	206	117
1 (stop-and-wait)	20%	5.352	93	257
4	0%	0.021	23,686	0
4	5%	0.532	939	100
4	10%	1.251	400	240
4	20%	2.233	224	420
16	0%	0.018	28,237	0
16	5%	0.680	735	501
16	10%	1.203	416	884
16	20%	2.364	212	1,708
64	0%	0.009	55,906	0
64	5%	0.633	790	1,731
64	10%	1.389	360	3,851
64	20%	2.371	211	6,528

> **Reading the 0% rows**
> With no loss, the transfers finished in about 0.01 to 0.02 seconds. That is too short to compare window sizes reliably, so those rows are ordinary timing noise and are left out of the analysis below.

6.2 Throughput

Figure 8 shows the main result. Moving from stop-and-wait to a window of 4 raised throughput by 2.0x at 5% loss, 1.9x at 10% and 2.4x at 20%. Windows of 16 and 64 gave no clear further gain.

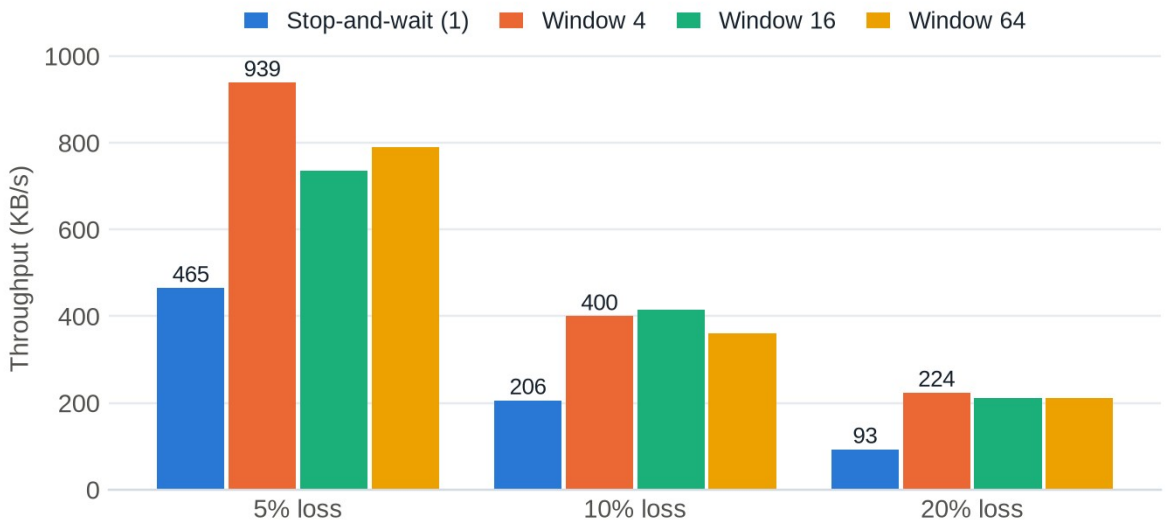


Figure 8 Throughput by window size and loss rate. 0% loss is omitted because those runs are too short to compare

Time here is dominated by waiting for timeouts rather than sending. For stop-and-wait at 20% loss, 257 timeouts of 20 ms account for 5.1 s of the 5.35 s total. A window appears to help because a single timeout can recover several packets at once, whereas stop-and-wait pays a full timeout for each one.

6.3 The cost of a large window

A larger window means more packets to resend after each timeout, and at high loss, a large window almost always contains a lost packet. Figure 9 shows that at 20% loss, window 64 retransmitted 6,528 packets to move 489. That is 14.35 packets sent for each packet delivered, compared with 1.86 for window 4, for no gain in speed.

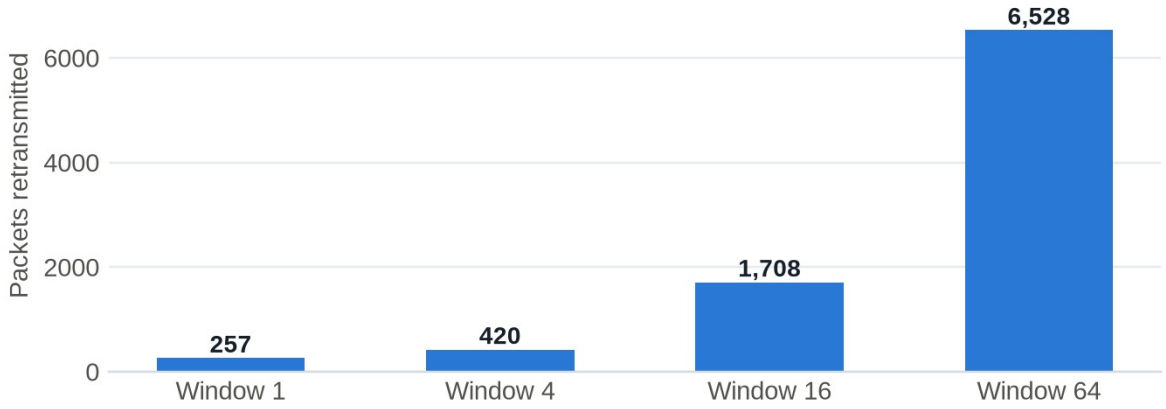


Figure 9 Packets retransmitted at 20% loss

6.4 Comparison with theory

Stop-and-wait is simple enough to predict. A packet succeeds only if the DATA packet and its ACK both survive, which has probability $(1 - p)^2$. The expected number of retransmissions for N packets is therefore:

$$\text{Expected retransmissions} = N \times \left(\frac{1}{(1 - p)^2} - 1 \right)$$

Table 9 Stop-and-wait, predicted against measured

Loss (p)	Success per try	Predicted	Measured	Difference
5%	0.9025	52.8	51	-3.5%
10%	0.8100	114.7	117	+2.0%
20%	0.6400	275.1	257	-6.6%

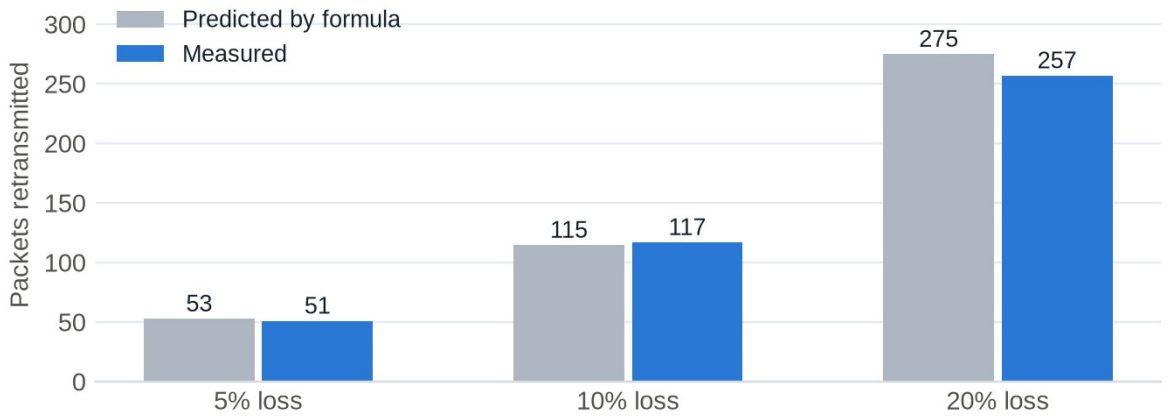


Figure 10 Predicted and measured retransmissions for stop-and-wait

The measured values are within 7% of the prediction. This is good evidence that both the protocol and the loss simulator behave as designed.

6.5 Limits of these results

> **What this experiment does not show**
> The tests ran on one machine, where round-trip delay is almost zero and loss is simulated with independent random drops. A sliding window's main benefit, keeping a link busy when ACKs take a long time to return, cannot appear in this setup. Real networks also lose packets in bursts. The best window size of 4 applies to this setup only, and should not be treated as a general rule.

7 Running the Project

7.1 Basic transfer

Python 3.9 or later is required, with no other setup. Start the server in one terminal and send a file from another:

Listing 6 Basic use

```
# Terminal 1: start the receiver
python3 server.py --out received

# Terminal 2: send a file
head -c 20000 /dev/urandom > sample.bin
python3 client.py sample.bin

# Prints nothing if the two files are identical
cmp sample.bin received/sample.bin
```

7.2 Demonstrating recovery

Restart the server with simulated loss and detailed logging, then watch the retransmissions on screen:

Listing 7 Lossy transfer with packet logs

```
python3 server.py --out received --loss 0.2 -v
python3 client.py sample.bin --window 4 --loss 0.2 --corrupt 0.05 -v
```

7.3 Other commands

Table 10 Command-line options

Option	Meaning
`--window N`	Packets in flight. 1 means stop-and-wait (client)
`--timeout S`	Seconds before resending (client)
`--loss P`	Chance of dropping each outgoing packet, for example 0.2
`--corrupt P`	Chance of damaging each outgoing packet
`--seed N`	Repeat the same simulated failures
`--host`, `--port`	Address to use. Use `--host 0.0.0.0` on the server to accept other machines
`-v`	Log every packet

Run the tests with `python3 -m unittest`, and regenerate the benchmark with `python3 benchmark.py > results.md`.

8 Limitations and Future Work

Table 11 Current limitations and how to address them

Limitation	Effect	Possible improvement
No encryption or authentication	Anyone who can reach the server can send it files, and same-named files are overwritten	Add TLS-style encryption and client authentication
No congestion control	The sender may overload a slow network	Adapt the window using loss, as TCP does
Fixed timeout	Too short wastes packets, and too long wastes time	Estimate it from round-trip times, as in RFC 6298
Go-Back-N resends good packets	Wasteful at high loss	Use selective repeat with a receiver buffer
One transfer at a time	A second client must wait	Track one session per client address
No resume, IPv4 only	An interrupted transfer restarts from the beginning	Keep progress on disk and support IPv6

The next experiment worth running is on a real link with delay, such as two laptops on a network or a simulated delay. That would show the benefit of the sliding window that the loopback tests could not.

9 Conclusion

This project designed and built RUFT, a working protocol that delivers files reliably over UDP. It defines its own packet format, session phases and recovery rules, and applies core Computer Networks concepts in running code: sequence numbers, cumulative acknowledgements, timeouts, sliding windows and error detection.

The testing supports these claims. All 22 tests pass, deliberately inserted bugs were caught, and a 2 MB file survived heavy simulated loss and corruption with not a single byte changed.

The measurements also gave two useful lessons. A window improved throughput by about two times under loss, but past a small size it only added retransmissions, so bigger is not always better. And a simple probability formula predicted stop-and-wait behaviour to within 7%, which shows how closely the theory matches a real implementation.

> **Project repository**
> https://github.com/PriyaRanjan018/ruft-protocol

References

[1] J. Postel, "User Datagram Protocol," RFC 768, IETF, Aug. 1980.

[2] W. Eddy, Ed., "Transmission Control Protocol (TCP)," RFC 9293, IETF, Aug. 2022.

[3] K. Sollins, "The TFTP Protocol (Revision 2)," RFC 1350, IETF, Jul. 1992.

[4] J. Iyengar and M. Thomson, Eds., "QUIC: A UDP-Based Multiplexed and Secure Transport," RFC 9000, IETF, May 2021.

[5] V. Paxson, M. Allman, J. Chu and M. Sargent, "Computing TCP's Retransmission Timer," RFC 6298, IETF, Jun. 2011.

[6] J. F. Kurose and K. W. Ross, Computer Networking: A Top-Down Approach, 8th ed. Pearson, 2021.

[7] A. S. Tanenbaum, N. Feamster and D. J. Wetherall, Computer Networks, 6th ed. Pearson, 2021.

[8] W. W. Peterson and D. T. Brown, "Cyclic Codes for Error Detection," Proceedings of the IRE, vol. 49, no. 1, pp. 228–235, Jan. 1961.

[9] National Institute of Standards and Technology, "Secure Hash Standard (SHS)," FIPS PUB 180-4, Aug. 2015.

[10] Python Software Foundation, "socket, struct, zlib and hashlib modules," The Python Standard Library documentation.